JUNIA

UE 11 - Veille et prospectives technologiques

# Déploiement et Sécurisation d'Architectures Cloud-Natives en Environnement On-Premise Air-Gapped

**Eudeline Nathan**

*Février 2026*

# 1 INTRODUCTION

La souveraineté des données et les exigences réglementaires strictes, telles que le Règlement Général sur la Protection des Données (RGPD), imposent aujourd'hui à de nombreuses infrastructures critiques de se détourner des services cloud publics. Le retour vers des environnements hébergés sur site (on-premise) et strictement isolés d'Internet, dits « air-gapped », s'accompagne d'un défi technologique majeur. En effet, l'écosystème cloud-native, reposant sur l'orchestrateur Kubernetes (The Kubernetes Contributors, 2026) et les pipelines de déploiement continu, présuppose par conception une connectivité réseau permanente pour le téléchargement d'images et la synchronisation des configurations.

Ce constat constitue le point de départ des travaux menés dans le cadre du présent projet d'alternance. Ce projet vise à concevoir une preuve de concept (PoC) d'un cluster Kubernetes de production, déployé de bout en bout sur des hyperviseurs locaux (Hyper-V) au sein d'un réseau hermétique. Le choix de ce sujet d'étude est motivé par un besoin industriel concret : allier la modernité, l'évolutivité et l'autocicatrisation des architectures cloud-natives avec les contraintes d'une isolation réseau totale et d'une stricte conformité réglementaire.

Cependant, reproduire l'expérience utilisateur et l'automatisation d'un fournisseur cloud au sein d'un réseau privé soulève une problématique opérationnelle complexe : **Comment garantir le déploiement automatisé, la reproductibilité de l'infrastructure et la sécurité d'architectures Kubernetes au sein d'environnements sur site strictement déconnectés, tout en s'affranchissant des processus manuels de transfert et de configuration, sources d'erreurs et de failles de sécurité ?**

Pour répondre à cette question, cette étude confronte tout d'abord les méthodes d'acheminement des dépendances logicielles en milieu isolé avec les standards émergents. Ensuite, elle analyse les stratégies d'automatisation de l'infrastructure et des configurations applicatives. Enfin, elle évalue les mécanismes de sécurisation réseau et d'exécution adaptés à ces plateformes restreintes.

# 2 SYNTHÈSE DE L'ÉTAT DE L'ART

## 2.1 Standardisation et Transfert d'Artefacts en Milieu Isolé

Le déploiement initial d'un cluster Kubernetes (bootstrap) et de son système d'exploitation sous-jacent hors-ligne nécessite de résoudre le problème fondamental de l'acheminement des dépendances. Dans le cadre de clusters virtualisés, la littérature et les retours d'expérience industriels soulignent la difficulté d'injecter des données d'initialisation (cloud-init) et des dépendances sans serveurs de métadonnées externes. Une fois l'OS instancié, Heckathorn et Trdina (Heckathorn & Trdina, 2025) relèvent que le transfert manuel d'images de conteneurs sous forme d'archives brutes (par exemple via des fichiers `.tar` chargés directement dans le

moteur d'exécution) est un processus chronophage, difficilement maintenable et propice aux erreurs de versionnage.

Pour y remédier de manière pérenne, la communauté scientifique préconise le déploiement de registres miroirs privés au sein de l'environnement isolé. L'efficacité de ces registres repose sur l'adoption de la spécification **Open Container Initiative** (OCI) (Open Container Initiative, 2025). Cette norme établit des formats standards pour les manifestes et l'agencement des images, permettant de traiter n'importe quel composant (conteneurs, chartes Helm, nomenclatures SBOM) comme un artefact unifié.

Grâce à cette standardisation, des outils open-source dédiés au monde air-gapped, tels que Hauler (The Hauler Contributors, 2026) ou Zarf (LF Projects, 2026), facilitent l'empaquetage de l'ensemble des dépendances dans des archives compressées uniques. Smith (Smith, 2021) démontre l'intérêt de ces outils lors du déploiement de logiciels sur des sous-marins nucléaires américains. L'utilisation de Zarf permet un transfert physique (sneaker-net) sécurisé par vérification de hachages cryptographiques (SHA), garantissant l'intégrité de la chaîne logicielle (Supply Chain) entre la zone connectée de préparation et la zone isolée de production.

## 2.2 Automatisation Déclarative et Maintien en Condition Opérationnelle

Au-delà de l'acheminement des fichiers, l'automatisation intégrale de l'infrastructure physique et de la couche applicative est indispensable pour garantir la résilience du système. L'approche **Infrastructure as Code** (IaC) est reconnue comme fondamentale pour prévenir les dérives de configuration (configuration drift). L'usage de fournisseurs Terraform (HashiCorp, 2026), même appliqués à des hyperviseurs sur site, couplé à des outils de gestion de configuration comme Ansible (Red Hat, 2026), permet de garantir la reproductibilité mathématique des environnements (Gheibi, 2025).

Une fois l'infrastructure provisionnée et le cluster instancié, le paradigme GitOps s'impose pour la gestion du cycle de vie applicatif. Gheibi (Gheibi, 2025) valide expérimentalement l'utilisation du contrôleur Argo CD (The Argo Contributors, 2026) au sein d'un cluster RKE2 totalement isolé. Le contrôleur GitOps, couplé à une instance GitLab hébergée en interne, assure une réconciliation continue entre l'état déclaré dans le dépôt de code (considéré comme l'unique source de vérité) et l'état réel du cluster. Cette boucle d'autocicatrisation (self-healing) s'opère de manière autonome, sans nécessiter d'accès à Internet.

Néanmoins, le passage à l'échelle de ces processus hors-ligne se heurte à des contraintes de latence lors du provisionnement. Les environnements sur site souffrent parfois de goulets d'étranglement lors de l'exécution de tâches distribuées sur de multiples nœuds. Des solutions d'optimisation, telles que l'intégration de bibliothèques réduisant la surcharge des connexions SSH (comme Mitogen pour Ansible (Wilson & NetworkGenomics, 2026)), sont identifiées pour pallier le surcoût de gestion des nœuds distribués en réseau local (Smith, 2021).

### 2.3 Sécurisation de l'Exécution, Réseau et Résilience

L'isolement réseau (air-gap) offre une barrière physique évidente contre les cyberattaques externes, mais ne protège pas la plateforme contre les vulnérabilités applicatives importées ou les menaces internes. L'application stricte des principes de moindre privilège (Role-Based Access Control) et l'isolation réseau inter-services sont considérées comme des fondements incontournables (Gheibi, 2025).

À ce titre, l'architecture réseau du cluster joue un rôle prépondérant. Les solutions traditionnelles basées sur le filtrage IP (`iptables`) atteignent leurs limites en termes de performance et d'observabilité. L'intégration de technologies avancées au sein du noyau Linux, telles que l'eBPF (Extended Berkeley Packet Filter) implémenté par des plugins réseau comme Cilium (Isovalent, 2026), offre une alternative moderne. L'eBPF permet un routage haute performance, une annonce directe des adresses IP sur les segments locaux (contournant le besoin d'équilibreurs de charge cloud) et une observabilité fine des flux, essentielle pour la détection d'anomalies en réseau restreint.

En outre, pour contrer un attaquant disposant de privilèges administrateur (ex: hyperviseur compromis au sein du datacenter), des approches matérielles sont explorées. L'intégration d'Environnements d'Exécution de Confiance (TEE) comme le framework Asylo permet de chiffrer les données directement en mémoire (Intel SGX) (Jamal Mahboob, 2021). Toutefois, cette approche alourdit drastiquement les temps de compilation et réduit la portabilité de l'architecture, entrant souvent en conflit avec les objectifs de standardisation et de flexibilité recherchés par les équipes de développement d'infrastructures sur site.

## 3 CONCLUSION ET JUSTIFICATION DES ORIENTATIONS TECHNIQUES

L'état de l'art confirme la viabilité technique du maintien de pratiques DevOps et cloud-natives modernes au sein d'infrastructures strictement déconnectées. Les travaux de recherche analysés valident formellement les fondations architecturales conçues et implémentées lors du projet d'alternance. En effet, l'utilisation du paradigme **Infrastructure as Code** (provisionnement Hyper-V via Terraform, configuration accélérée par Ansible et Mitogen) ainsi que l'adoption du modèle GitOps (ArgoCD en environnement air-gapped gérant des services eBPF comme Cilium) s'inscrivent parfaitement dans les meilleures pratiques de résilience identifiées par la communauté scientifique.

Cependant, la confrontation de cette architecture avec la littérature met en lumière une marge d'amélioration significative concernant la chaîne d'approvisionnement des artefacts. L'acheminement des dépendances reposait initialement sur un rôle d'automatisation personnalisé manipulant des archives `.tar` brutes pour les injecter localement dans les moteurs de conteneurs. La littérature démontre que cette méthode est vieillissante face aux nouveaux standards. Les outils conformes à la spécification OCI (tels que Hauler ou Zarf) offrent une

approche nettement supérieure, permettant une gestion unifiée, cryptographiquement vérifiée et versionnée de l'ensemble des dépendances d'un cluster.

Par conséquent, les travaux futurs menés en entreprise devront se concentrer sur la refonte de la **Supply Chain** logicielle du cluster. L'objectif sera de déprécier les scripts de transfert d'archives personnalisés au profit d'une intégration complète des standards OCI. Cette évolution, couplée à l'exploitation approfondie de l'observabilité offerte par eBPF, permettra de consolider la sécurité et la reproductibilité de la plateforme, répondant ainsi pleinement aux exigences de souveraineté et de robustesse imposées par le cadre de l'alternance.

## BIBLIOGRAPHIE

The Argo Contributors. (2026, ). *Argo CD - Declarative GitOps CD for Kubernetes*. https://argo-cd.readthedocs.io/

Gheibi, M. (2025). *Challenges in applying cloud cutting edge technologies in air gapped on premises datacenters* [Master's thesis].

HashiCorp. (2026, ). *Terraform by HashiCorp - Infrastructure as Code*. https://www.terraform.io/

The Hauler Contributors. (2026, ). *Hauler: The Airgap Swiss Army Knife*. https://docs.hauler.dev/

Heckathorn, M., & Trdina, M. (2025). *Kubernetes (k8s) in the Air Gap* [Technical report].

Isovalent. (2026, ). *Cilium: eBPF-based Networking, Observability, and Security*. https://cilium.io/

Jamal Mahboob, J. C. (2021, ). A Kubernetes CI/CD Pipeline with Asylo as a Trusted Execution Environment Abstraction Framework. *2021 IEEE 11th Annual Computing and Communication Workshop and Conference (CCWC)*.

The Kubernetes Contributors. (2026, ). *Kubernetes: Production-Grade Container Orchestration*. https://kubernetes.io/

LF Projects. (2026, ). *Zarf: DevSecOps for Air Gap & Disconnected Environments*. https://zarf.dev/

Open Container Initiative. (2025, ). *Open Container Initiative Distribution Specification*. https://github.com/opencontainers/distribution-spec

Red Hat. (2026, ). *Ansible is Simple IT Automation*. https://www.ansible.com/

Smith, B. A. (2021). *A DevSecOps Approach for Developing and Deploying Containerized Cloud-Based Software on Submarines* [Master's thesis].

Wilson, D., & NetworkGenomics. (2026, ). *Mitogen for Ansible - Faster Execution*. https://mitogen.networkgenomics.com/ansible_detailed.html